

Capacitação Back-end — roteiros

AUTOMIC JR. · QUATRO ENCONTROS

Sumário

Roteiros — índice para quem apresenta

O número honesto

As marcações nos slides

Regra geral para o dia

Roteiro — Aula 1: back-end, Ruby e o primeiro Rails

Antes de começar

Cronograma

Bloco a bloco

00:00 — Abertura (slides 1–2)

00:05 — Ferramentas e setup (3–10)

00:28 — O que é back-end (11–15)

00:40 — Rede (16–19)

00:53 — HTTP (20–27)

01:15 — API REST (28–29)

01:35 — Ruby (30–45)

02:17 — Prática 1 (46)

02:27 — Rails (47–58)

02:52 — Prática 2 (59)

03:22 — Fecho (60–62)

Perguntas que vão aparecer

Dever de casa

Roteiro — Aula 2: banco, ActiveRecord e o model User

Antes de começar

Cronograma

Bloco a bloco

00:00 — Retomada (1–3)

00:06 — Banco (4–9)

00:24 — ActiveRecord (10–15)

00:42 — Migrations (16–19)

01:02 — Senha (20–25)

01:32 — Validações (26–29)

01:47 — Associações (30–36)

02:09 — Concerns (37–40)

02:24 — Console e testes (41–45)

02:41 — Prática (46)

Perguntas que vão aparecer

Dever de casa

Roteiro — Aula 3: as rotas de autenticação

A decisão que você precisa tomar antes

Opção A — código pronto, leitura guiada (recomendada)

Opção B — digitar junto

Antes de começar

Cronograma (Opção A)

Bloco a bloco

00:00 — O caminho (1–3)

00:06 — Arquitetura (4–13)

00:34 — Cadastro (14–16)

00:46 — E-mail (17–20)

01:00 — JWT (21–28)

01:36 — Login (29–36)

02:04 — Logout (37–43)

02:28 — Recuperação de senha (44–49)

03:01 — Prática (55)

Perguntas que vão aparecer

Dever de casa

Roteiro — Aula 4: VPS, Docker, Kamal e deploy

O que preparar com antecedência

Antes de começar

Cronograma

Plano B, decidido de véspera

Bloco a bloco

00:00 — VPS (1–7)

00:16 — A VM (8–14)

00:51 — Ubuntu (15–21)

01:33 — Docker (22–26)

01:49 — Kamal (27–35)

02:15 — CI/CD e OIDC (36–44)

03:00 — TLS (45–52)

03:22 — O primeiro deploy (53–56)

03:52 — Fecho (57–61)

Perguntas que vão aparecer

Depois da capacitação

Roteiros — Índice para quem apresenta

Um runbook por encontro, com cronograma, o que falar em cada bloco, as demos ao vivo, as perguntas que a turma faz e o plano B:

	Roteiro	Slides	Estimado	Disponível
1	Back-end, Ruby e o primeiro Rails	62	3h27	3h
2	Banco, ActiveRecord e o model <code>User</code>	48	3h20	3h
3	As rotas de autenticação	57	3h30 *	3h
4	VPS, Docker, Kamal e deploy	61	4h02 *	3h

* já contando o plano recomendado. Digitando todo o código da Aula 3 seriam ~6h30; fazendo o pipeline OIDC completo na Aula 4, ~4h45.

O número honesto

São ~14h de conteúdo em 12h de encontro. Cada roteiro traz os cortes específicos, em ordem de prioridade, e um **ponto de decisão** com relógio: se às tantas horas a turma não estiver em tal lugar, faça tal coisa.

As duas decisões que resolvem a maior parte do problema:

1. **Aula 3 — não digitem o código.** `git checkout aula-03` no começo e leitura guiada dos arquivos projetados. A prática vira modificar o que existe. **Economiza ~3h30.**
2. **Aula 4 — prepare o que não ensina.** Registros DNS e um certificado Origin CA wildcard, feitos na véspera. **Economiza ~40 min.** Se ainda assim não couber, o roteiro traz dois planos B.

Base do cálculo: slide a ~2 min, digitação guiada a ~5 linhas/min, cliques em portal cronometrados no percurso real.

As marcações nos slides

Marca	Significa
# CORTÁVEL	Pré-requisito. Corte se a turma já sabe Git, terminal, SQL ou Linux.
# EXTRA	Conceito de fundo (rede, TLS, XSS...). O exercício funciona sem ele — mas é o que separa "copiei" de "entendi".

```
grep -n "CORTÁVEL\|EXTRA" slides/conteudo/aula-0*.yaml
```

Cortar todos os `# EXTRA` devolve ~45 slides, uns 90 minutos no total das quatro aulas. **A recomendação é o contrário:** mantenha os `# EXTRA` e economize no código, com a decisão 1 acima.

Regra geral para o dia

Quando atrasar, corte **conteúdo**, nunca **prática**. Uma turma que viu 40 slides e fez a coisa funcionar aprendeu mais que uma que viu 60 e foi embora com o erro na tela.

Roteiro — Aula 1: back-end, Ruby e o primeiro Rails

Deck: `slides/build/aula-01-fundamentos-de-back-end.pptx` (62 slides) **Apostila da turma:** `01-fundamentos.md` · **Checkpoint:** `aula-01`

Antes de começar

Na semana anterior

- [] Mandar `00-preparacao.md` no grupo e cobrar resposta de quem travou.
- [] Levantar quem usa Windows → WSL2 instalado **antes**.
- [] Cobrar a conta Azure for Students (a verificação demora dias e é da Aula 4).

No dia, antes de a turma chegar

- [] Deck aberto, modo apresentador (as notas de cada slide estão preenchidas).
 - [] Dois terminais grandes: um para o `irb`, outro para o `rails`.
 - [] Fonte do terminal em pelo menos 18pt.
 - [] Insomnia aberto.
 - [] `curl -i https://api.seemxxiii.tech/api/v1/status` testado — é a sua demo do slide 26.
 - [] `git checkout aula-01` num diretório separado, pronto para socorrer quem travar.
-

Cronograma

Relógio	Slides	Bloco	Min
00:00	1–2	Abertura e objetivos	5
00:05	3–10	Ferramentas e setup	23
00:28	11–15	O que é back-end	12
00:40	16–19	Rede: servidor, IP, porta, DNS, URL	13
00:53	20–27	HTTP e JSON	22
01:15	28–29	API REST	10
01:25	—	Intervalo	10
01:35	30–45	Ruby para quem sabe Python	42
02:17	46	Prática 1 — <code>irb</code>	10
02:27	47–58	Rails e a primeira rota	25
02:52	59	Prática 2 — <code>GET /api/v1/status</code>	30
03:22	60–62	Git, recapitulação, fim	5

Isso dá 3h27. Ou você corta, ou aceita passar. Onde cortar, em ordem:

1. Slide 6 (`O TERMINAL`) e 60 (`GIT`) — os dois marcados `# CORTÁVEL` . **–7 min**
2. Slides 40–43 (argumentos nomeados, `Struct` , exceções) — explique quando aparecerem na Aula 3. **–12 min**
3. Slide 18 (`DNS`) — volta na Aula 4 de qualquer jeito. **–3 min**

Cortando os três, fecha em 03:05.

Bloco a bloco

00:00 — Abertura (slides 1–2)

Você se apresenta e diz de onde veio a capacitação: **é a continuação da do Fiuza**. Lá foi o que o usuário vê; aqui é o outro lado.

Nos objetivos, a frase que prende: *"no fim do encontro 4, cada um de vocês vai ter uma URL `https://` própria, funcionando, que qualquer um do mundo consegue chamar."*

00:05 — Ferramentas e setup (3–10)

Passe rápido pelos slides e **pare no 10**. Todo mundo roda os quatro comandos ao mesmo tempo:

```
curl https://mise.run | sh
mise use --global ruby@3.4.9
gem install rails -v 8.1.3
docker run --rm hello-world
```

Peça para todos mostrarem a saída de `ruby -v` antes de seguir.

Ponto de não-retorno: se passou de 00:35 e ainda tem gente instalando, **pare**. Pareie quem travou com quem terminou e siga. Instalação é problema de casa, não de aula.

00:28 — O que é back-end (11–15)

O slide 12 é a ponte com a capacitação anterior: mostre a coluna do front e diga "isso vocês já viram". O slide 13 tem o ponto que importa: **tudo que roda no navegador o usuário consegue ler e alterar** — por isso a validação que vale é a do back.

O restaurante (14) funciona bem. Volte nele quando falar de API.

00:40 — Rede (16–19)

Aqui muita gente descobre coisa que usava sem saber.

- **16:** servidor não é máquina especial, é programa esperando numa porta.
- **17:** a tabela de portas. Diga que 22, 80 e 443 voltam na Aula 4, quando forem abrir e fechar firewall na mão.
- **19:** desmonte a URL na tela. Pergunte: "por que `localhost:3000` tem dois pontos e `google.com` não?" Deixe alguém responder.

00:53 — HTTP (20–27)

Slide 21 é o coração do bloco. Uma requisição HTTP é texto puro. Se der, faça ao vivo:

```
curl -v https://api.seemxxiii.tech/api/v1/status
```

O `-v` mostra as linhas com `>` (o que foi) e `<` (o que voltou). É a mesma coisa do slide, acontecendo de verdade.

No **23** (`Content-Type`), avise: "esquecer esse cabeçalho é o erro nº 1 de vocês na Aula 3. Vem 400 e a mensagem não ajuda em nada."

No **25** (status codes), a pergunta: "qual a diferença entre 401 e 403?" — 401 é "quem é você?", 403 é "sei quem você é, e você não pode". Diga que cai em entrevista.

No **fim do bloco**, marque a frase: **HTTP não tem memória**. Escreva no quadro se tiver. É o problema inteiro da Aula 3.

01:15 — API REST (28–29)

Curto. O que fica: o verbo é a ação, o endereço é a coisa. `/criarPalestra` está errado; `POST /lectures` está certo.

O 29 explica por que o SEEM é uma API: um back-end servindo o app e o painel.

01:35 — Ruby (30–45)

O bloco mais longo. **Não leia os slides — rode no `irb` ao vivo.**

```
3.class          # Integer
"oi".class       # String
:oi.class        # Symbol
nil.class        # NilClass

3.times { puts "oi" }

usuario = { nome: "Ana", role: :admin }
usuario[:nome]

[1, 2, 3].map { |n| n * n }
[1, 2, 3].select { |n| n.odd? }
```

Pontos onde vale parar:

- **33** — as três diferenças: `end`, `if` no fim da linha, retorno implícito.
- **35** — símbolo não existe em Python. A regra: conteúdo que o usuário lê é string; nome de coisa no código é símbolo.
- **38** — `map` / `select` são iguais aos de Python. A diferença é que aqui são o caminho normal.
- **39** — o `?` e o `!`. Diga que o Rails inteiro depende dessa convenção.
- **40–41** — argumentos nomeados e `Struct`. Fale a frase: "o `user:` sozinho não é erro de digitação", porque eles vão ver isso em todo service da Aula 3.

02:17 — Prática 1 (46)

Dez minutos no `irb`. Circule pela sala. Quem terminar rápido, mande fazer o bônus (`map` para elevar ao quadrado).

02:27 — Rails (47–58)

- **49** é a ideia central: você não configura o óbvio, você segue o combinado.
- **51** (Django × Rails) — pergunte quem já usou Django ou Flask e ancore neles.
- **54** (Zeitwerk) — a frase: "não é estilo, é mecanismo. Errou o caminho, a classe não existe."
- **58** — mostre a rota e o controller lado a lado, e aponte a convenção ligando os dois.

02:52 — Prática 2 (59)

A entrega da aula. **Ninguém sai sem os 200 na tela.**

```
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
docker compose up -d
bin/rails db:prepare
bin/rails server
```

Depois a rota, e o teste no Insomnia.

Quem travar: `git checkout aula-01`.

03:22 — Fecho (60–62)

Recapitule em cinco frases (slide 61) e feche com o que vem: *"na próxima, a API ganha memória."*

Perguntas que vão aparecer

Pergunta	Resposta curta
"Por que Ruby e não Python?"	Porque o <code>seem-backend</code> é Ruby, e porque Rails entrega um sistema inteiro sem você montar peça por peça. E a linguagem é o de menos: o que vocês vão aprender aqui vale em qualquer uma.
"Rails não morreu?"	GitHub, Shopify e Basecamp rodam nele hoje. O que morreu foi o hype, não a ferramenta.
"Preciso decorar os status codes?"	Não. Precisa saber a faixa: 2xx deu certo, 4xx você errou, 5xx eu errei. O resto se consulta.
"Por que não usar o navegador para testar a API?"	O navegador só sabe fazer GET. Nossas rotas são POST e DELETE.
"Posso usar o Ruby que já está instalado no meu Linux?"	Pode dar certo e pode dar errado. É por isso que existe o mise: a versão exata, por projeto.
"O <code>--api</code> tira alguma coisa que vou precisar?"	Tira view, sessão de navegador e assets. Se um dia precisar, dá para trazer de volta — foi o que fizemos com os cookies.

Dever de casa

1. Terminar a prática, se não deu tempo.
2. Ler `ruby-para-pythonistas.md` inteiro.
3. Bônus do slide 59: fazer a rota devolver `RUBY_VERSION` e `Rails.version`.

Roteiro — Aula 2: banco, ActiveRecord e o model `User`

Deck: `slides/build/aula-02-banco-de-dados-e-activerecord.pptx` (48 slides) **Apostila da turma:** `02-activerecord.md` · **Checkpoint:** `aula-02`

É a aula mais tranquila das quatro em termos de tempo. Use a folga para o console ao vivo — é o que mais fixa.

Antes de começar

- [] `git checkout aula-01` num diretório de socorro, com o banco já preparado.
 - [] `docker compose up -d` rodando no seu, para o console não travar na hora.
 - [] Terminal grande, com o `bin/rails console` já aberto e testado.
 - [] Ter em mãos um exemplo real de vazamento de senha para citar (LinkedIn 2012, 6,5 milhões de hashes SHA-1 sem salt — quebrados em dias).
 - [] Conferir quem não terminou a prática da Aula 1 e resolver nos primeiros 10 minutos.
-

Cronograma

Relógio	Slides	Bloco	Min
00:00	1–3	Abertura e retomada	6
00:06	4–9	Banco, transação, Postgres, container	18
00:24	10–15	ActiveRecord, ActiveSupport, <code>blank?</code>	18
00:42	16–19	Migrations e <code>schema.rb</code>	20
01:02	20–25	Senha: hash, bcrypt, <code>has_secure_password</code>	20
01:22	—	Intervalo	10
01:32	26–29	Validações e normalização	15
01:47	30–36	Associações e N+1	22
02:09	37–40	Concerns	15
02:24	41–45	Console ao vivo, seeds, fixtures, teste	17
02:41	46	Prática	35
03:16	47–48	Recapitulação e fim	4

Dá 3h20. Cortes possíveis:

- Slide 5 (`O MODELO RELACIONAL` , marcado `# CORTÁVEL`) se a turma já viu SQL. **–5 min**
- Slide 36 (`A ARMADILHA N+1`) — fica na apostila. **–5 min**
- Encurtar a prática para 25 min e mandar o resto de casa. **–10 min**

Bloco a bloco

00:00 — Retomada (1–3)

O slide 3 é a ponte: *"temos uma API que responde. Ela não guarda nada. Reiniciou, esqueceu tudo. Hoje ela ganha memória."*

00:06 — Banco (4–9)

Transação (6–7) é o slide que eles vão precisar na Aula 3. Use o exemplo do dinheiro: debitar de um e creditar no outro têm que ser a mesma operação; debitar sozinho é dinheiro que sumiu.

Aponte o `!` no slide 7 e explique: dentro da transação você **quer** que estoure, porque `save` devolve `false` em silêncio e a transação seguiria feliz gravando metade.

No 8, a frase: *"desenvolver no mesmo banco da produção elimina uma categoria inteira de bug."*

00:24 — ActiveRecord (10–15)

O slide 13 é o que causa espanto em quem vem de Django: **a classe não declara nada**.

```
class User < ApplicationRecord
end
```

Diga: *"isso já tem todas as colunas. A fonte da verdade é a migration, não a classe."*

15 é para rodar no console, não para ler:

```
nil.blank?      # true
"".blank?       # true
" ".blank?      # true
0.blank?        # false    ← aqui alguém vai reclamar
```

E a pegadinha que vem de Python: em Ruby, `if 0` executa. `if ""` executa. Só `nil` e `false` são falsos.

00:42 — Migrations (16–19)

Gere uma migration ao vivo:

```
bin/rails generate migration CreateUsers
```

Mostre o arquivo criado, o timestamp no nome, e diga que é ele que define a ordem.

O slide 18 tem o ponto mais importante do bloco: índice único é restrição do banco, validação é mensagem bonita. Duas requisições simultâneas passam pela validação juntas — as duas consultam, as duas não acham ninguém — mas só uma vence o índice.

No 19, a regra: **nunca edite migration que já rodou em produção**. Crie outra.

01:02 — Senha (20–25)

O bloco mais importante da aula inteira.

- **21** — comece pelo susto. Cite o vazamento que você separou.
- **22** — hash não é criptografia. Criptografia tem volta; hash não.
- **23** — por que bcrypt e não SHA-256: SHA é rápido, **e é esse o problema**. GPU testa bilhões por segundo. bcrypt é lento de propósito, com custo ajustável.
- **24** — desmonte o hash na tela: versão, custo, salt, digest.

Demonstre no console — é o momento em que a ficha cai:

```
BCrypt::Password.create("senha123")
BCrypt::Password.create("senha123")  # rode DE NOVO
```

São diferentes. É o salt. Pergunte por que, antes de responder.

01:32 — Validações (26–29)

O 29 (normalizar antes de validar) fecha com o 18: se você não normaliza, o índice único não serve para nada, porque o banco acha que `Ana@UFOP.br` e `ana@ufop.br` são valores diferentes.

01:47 — Associações (30–36)

Bloco novo, e é o que eles mais vão usar no primeiro projeto de verdade.

A regra que resume tudo (slide 32): **a chave estrangeira fica no lado "muitos"**. Quem carrega a coluna usa `belongs_to`; quem é apontado usa `has_many`.

No console:

```
ana = User.first
ana.login_events.create!(client: "mobile", occurred_at: Time.current)
ana.login_events.count
ana.login_events.recentes.first.user.name    # e volta
```

Demonstre o N+1 de verdade (36) — com o log na tela:

```
User.all.each { |u| puts u.login_events.count }    # conte as consultas
User.includes(:login_events).each { |u| puts u.login_events.count }
```

A diferença no log é o argumento. Nenhum slide convence tanto.

02:09 — Concerns (37–40)

A ponte com a Aula 1: *"lembram do módulo que se inclui numa classe? Isto aqui é ele, com nome de Rails."*

O slide 40 tem as três decisões — a que mais rende é a terceira: `email_verified_at` é data, não booleano, porque um dia alguém vai abrir chamado perguntando *quando* a conta foi ativada.

02:24 — Console e testes (41–45)

O slide 45 (`authenticate_by_email`) merece atenção: o ataque de temporização. Se a resposta volta em 1ms quando o e-mail não existe e em 100ms quando existe, dá para descobrir quem tem conta cronometrando. Guarde — volta na Aula 3.

02:41 — Prática (46)

Circule. Os pontos onde travam:

- esqueceram `db:migrate` depois de criar a migration;
- escreveram `has_secure_password` sem adicionar a gem `bcrypt` no Gemfile;
- `matricula` com máscara (`20.112-34`) falhando na validação — é exatamente o motivo de `normalize_matricula` existir.

Perguntas que vão aparecer

Pergunta	Resposta curta
"Por que não SQLite? É mais simples."	Uma escrita por vez no banco inteiro. Num servidor com várias requisições, trava.
"Dá para descriptografar o <code>password_digest</code> ?"	Não. Não é criptografia, é hash: caminho só de ida. Nem você consegue ver a senha do usuário — e isso é a intenção.
"Por que o bcrypt é lento? Isso não é ruim?"	É lento de propósito . 100ms para você é nada; para quem tenta bilhões de senhas, é o que inviabiliza.
"Se eu já valido no model, para que o índice único?"	Concorrência. Duas requisições ao mesmo tempo passam pelas duas validações e só uma vence o índice.
"Por que não <code>t.boolean :email_verified?</code> "	Porque booleano responde "sim" e data responde "sim, dia 12 às 14h". Um dia alguém vai perguntar quando.
"Posso apagar uma migration antiga e refazer?"	Na sua máquina, sim. Depois que rodou em produção, nunca — crie outra.
" <code>dependent: :delete_all</code> ou <code>:destroy</code> ?"	<code>delete_all</code> é um <code>DELETE</code> só, direto no banco, e não roda callback. <code>:destroy</code> carrega cada registro e roda os callbacks. Aqui não há callback, então <code>delete_all</code> é mais rápido.

Dever de casa

1. Terminar a prática.
2. Bônus: escrever o N+1 de propósito, contar as consultas no log e consertar com `includes`.
3. Ler a seção "Uma sutileza de segurança" da apostila — ela é o começo da próxima aula.

Roteiro — Aula 3: as rotas de autenticação

Deck: `slides/build/aula-03-autenticacao.pptx` (57 slides) **Apostila da turma:** `03-autenticacao.md`

• **Checkpoint:** `aula-03`

Esta é a aula que não cabe. São 1396 linhas de código em 37 arquivos. Digitando, dá 6h30. Leia a seção seguinte antes de qualquer outra coisa.

A decisão que você precisa tomar antes

Você tem duas formas de conduzir esta aula:

Opção A — código pronto, leitura guiada (recomendada)

No começo da aula, todo mundo roda:

```
git checkout aula-03
bin/rails db:prepare
```

Você percorre os arquivos **projetados**, explicando decisão por decisão. Eles acompanham no editor. A prática vira **modificar** o que já existe — que é o que se faz num emprego de verdade.

Cabe em ~2h50. É o que este roteiro assume.

Opção B — digitar junto

Só funciona se você tiver 6h ou dividir em dois encontros. Se for por aqui, corte para **três rotas**: cadastro, login e logout. Recuperação de senha vira leitura da apostila.

Antes de começar

- [] `git checkout aula-03` testado na sua máquina, com `bin/rails test` verde.
- [] Servidor rodando e o Insomnia com as cinco requisições já montadas — você vai fazer o fluxo completo ao vivo duas vezes.
- [] jwt.io aberto numa aba.
- [] Um token válido copiado, pronto para colar no jwt.io.
- [] `tmp/letter_opener/` limpo, para o e-mail da demo ser o primeiro da lista.
- [] Editor com `app/services/` e `app/controllers/api/v1/` já abertos na barra lateral.

Cronograma (Opção A)

Relógio	Slides	Bloco	Min
00:00	1–3	Abertura e o caminho de uma requisição	6
00:06	4–13	Arquitetura: service, serializer, erro, filtro, middleware	28
00:34	14–16	Cadastro e enumeração de usuários	12
00:46	17–20	Mailer, letter_opener, <code>deliver_now</code>	14
01:00	21–28	Sessão, JWT, Base64, assinatura	26
01:26	—	Intervalo	10
01:36	29–36	Login, cookie, XSS, CSRF, 401×403	28
02:04	37–43	Logout: denylist e <code>token_version</code>	24
02:28	44–49	Recuperação de senha e a transação	18
02:46	50–54	Testes, ferramentas, CORS	15
03:01	55	Prática	25
03:26	56–57	Recapitulação e fim	4

Dá 3h30 mesmo na Opção A. Cortes, em ordem:

1. Slides 33–34 (XSS e CSRF) — o material fica na apostila. **–8 min**
2. Slides 53–54 (ferramentas e CORS) — mostre rodando e siga. **–8 min**
3. Slides 17–20 (mailer) — mostre só o e-mail abrindo no navegador. **–8 min**

Nunca corte o bloco 37–43. O logout é o ponto alto da aula, e é o que diferencia esta capacitação de um tutorial de YouTube.

Bloco a bloco

00:00 — O caminho (1–3)

O slide 3 é o mapa da aula inteira. Deixe ele na tela enquanto fala a regra:

Controller recebe requisição e devolve resposta. Regra de negócio não mora nele.

00:06 — Arquitetura (4-13)

Abra `app/services/users/register.rb` projetado e conte as seis coisas que ele faz. Depois pergunte: "quanto disso vocês conseguiriam testar sem subir uma requisição HTTP inteira?"

- **7** — serializer. Faça a demonstração do susto: "o que acontece se eu escrever `render json: user`?"
Mostre no console:

```
ruby User.first.as_json.keys
```

Está lá o `password_digest` e os digests dos códigos. **Uma linha, e vazou.**

- **10-12** — `before_action` e middleware. O slide 12 é o mapa completo. Se der, rode:

```
bash bin/rails middleware
```

- **13** — strong parameters. A frase: "sem isso, alguém manda `role: admin` no cadastro e vira admin. Isso tem nome: mass assignment, e já derrubou sistema grande."

00:34 — Cadastro (14-16)

O slide 16 é o primeiro dos quatro momentos de "enumeração" da aula. Marque isso: você vai voltar nele três vezes, e no fim eles devem conseguir prever a decisão sozinhos.

Pergunta para a turma antes de virar o slide: "o e-mail já existe. O que a API deve responder?" Alguém vai dizer "este e-mail já está cadastrado". Aí você mostra por que não.

00:46 — E-mail (17-20)

Faça o cadastro ao vivo no Insomnia e **mostre o e-mail abrindo no navegador** pelo letter_opener. É um daqueles momentos em que a turma acorda.

No slide 20, a decisão contra o livro-texto: `deliver_now` porque, numa fila, o código de 6 dígitos ficaria gravado **em texto** na tabela de jobs.

01:00 — JWT (21-28)

O bloco conceitual mais denso. Comece pelo problema (22): "lembram que HTTP não tem memória? Então como o servidor sabe que vocês já entraram?"

Faça a demo do jwt.io. Cole o token que você preparou e mostre o payload legível na tela.

"Está assinado. Não está criptografado. Qualquer um lê. Então nunca ponham aqui nada que não possa ser lido."

No **28**, o `true` do `JWT.decode`. Diga que já foi CVE em várias bibliotecas, e que eles vão brincar com isso na prática.

01:36 — Login (29–36)

- **30** — os dois transportes. O `client` no corpo existe por isso.
- **31–32** — o que é cookie, e as três flags.
- **33–34** — XSS e CSRF. Se o tempo apertar, é aqui que você corta.
- **36** — o segundo "enumeração": mesma mensagem para e-mail inexistente e senha errada. E o `DUMMY_PASSWORD_DIGEST` da Aula 2 volta, fechando o buraco pelo lado do tempo.

02:04 — Logout (37–43)

O ponto alto. Conduza como um problema, não como uma solução.

1. Pergunte: *"o token vale 24h e o servidor não guarda sessão. O que 'sair' significa?"*
2. Deixe a turma propor. Alguém vai dizer "apaga no cliente". Aceite e pergunte: *"e se alguém já copiou o token?"*
3. Aí você apresenta a denylist (40).
4. O slide 41 é a sacada: o TTL de cada entrada é o que faltava para o token expirar, então **a lista se limpa sozinha**.
5. E o 42 mostra a outra ferramenta: `token_version` derruba tudo de uma vez.

Demonstre ao vivo. Vale mais que os seis slides:

```
TOKEN=... # pegue do login
curl $API/me -H "Authorization: Bearer $TOKEN" # 200
curl -X DELETE $API/sessions -H "Authorization: Bearer $TOKEN"
curl -i $API/me -H "Authorization: Bearer $TOKEN" # 401
```

02:28 — Recuperação de senha (44–49)

Terceiro e quarto "enumeração" (46). A essa altura, **pergunte antes**: *"o e-mail não existe. O que respondemos?"* Eles devem acertar sozinhos.

O slide 48 amarra com a Aula 2: a transação. E o 49 tem o ponto do fluxo inteiro —

`invalidate_sessions!`: *"se alguém invadiu a conta, trocar a senha tem que expulsar o invasor. Sem essa linha, ele continua logado."*

Demonstre: faça o reset e mostre o token antigo virando 401.

03:01 — Prática (55)

Na Opção A, a prática é **modificar**:

1. Fluxo completo no Insomnia (obrigatório).
2. Colar o próprio token no jwt.io e achar `sub`, `jti` e `exp`.
3. **Bônus 1:** `PATCH /api/v1/me` para editar só o `name`.

4. **Bônus 2:** trocar o `true` do `JWT.decode` por `false`, forjar um token com outro segredo, ver `/me` responder 200 — e depois desfazer. É o exercício que mais marca.

Perguntas que vão aparecer

Pergunta	Resposta curta
"Se o payload é público, não é inseguro?"	Não, desde que você não coloque segredo lá. O token prova quem você é; ele não precisa esconder isso de você.
"Por que não usar sessão como todo mundo?"	Porque o cliente principal é app nativo, que não tem cookie. E porque assim qualquer servidor valida sozinho, sem sessão compartilhada.
"Então JWT é pior que sessão?"	É diferente. Sessão facilita o logout e complica o escalar; token faz o contrário. Nós resolvemos o logout com a denylist.
"A denylist não é uma sessão disfarçada?"	Um pouco — mas só guarda os tokens revogados, não todos, e expira sozinha. É bem mais barata.
"Por que 15 minutos no código, e não 1 hora?"	Janela curta reduz o tempo em que um código interceptado serve. 15 min é o suficiente para checar e-mail sem correr.
"E se o e-mail do usuário estiver errado no cadastro?"	Ele nunca confirma, nunca loga, e a conta fica órfã. É por isso que existe o <code>resend</code> .
"Posso usar Devise em vez de escrever tudo isso?"	Pode, e em projeto de verdade normalmente é o que se faz. Mas você não entenderia nada do que ele faz — e é isso que estamos aprendendo.
"Por que <code>unprocessable_content</code> e não <code>unprocessable_entity</code> ?"	Mesmo código 422. O nome foi renomeado na especificação, e o Rails 8 seguiu.

Dever de casa

1. Fazer os dois bônus.
2. Ler `app/services/users/complete_password_reset.rb` inteiro e explicar, por escrito, por que a política de senha é validada **antes** de consumir o código.
3. Rodar `bin/brakeman` e ler o que ele procura.

Roteiro — Aula 4: VPS, Docker, Kamal e deploy

Deck: `slides/build/aula-04-vps-docker-kamal-e-deploy.pptx` (61 slides) **Apostila da turma:** `04-deploy.md` · **Checkpoint:** `aula-04`

A aula mais arriscada das quatro. O gargalo não é digitar, é clicar em portal — e portal falha. Leia a seção seguinte antes de qualquer coisa.

O que preparar com antecedência

Estas quatro coisas **não ensinam nada ao serem feitas doze vezes**, e devolvem ~40 minutos:

#	O que	Quando
1	Registros DNS <code>aluno1.capacita...</code> <code>alunoN.capacita</code> já criados no seu Cloudflare, apontando para IP <code>0.0.0.0</code> (eles trocam pelo IP da VM deles)	véspera
2	Um certificado Origin CA wildcard <code>*.capacita.<seu-domínio></code> , com o <code>.pem</code> e a chave prontos para distribuir	véspera
3	Verificar que todo mundo já tem crédito Azure aparecendo no portal	uma semana antes
4	Uma conta Resend com o domínio verificado, e uma API key para a turma	véspera

E o mais importante:

Faça o percurso inteiro sozinho uma vez, com uma VM descartável, antes do encontro. É a única forma de descobrir onde a sua assinatura vai reclamar de cota. Se você não fizer isso, a aula vai parar num erro que você está vendo pela primeira vez, na frente de todo mundo.

Antes de começar

- [] Portal Azure aberto e logado, numa aba anônima (para não vazar recursos do SEEM na projeção).
- [] Cloudflare aberto no domínio da capacitação.
- [] GitHub do repositório da capacitação aberto em Settings → Secrets and variables.
- [] Sua VM de teste **destruída**, para você fazer o percurso junto com eles do zero.
- [] `docs/troubleshooting.md` aberto numa aba — você vai usar.
- [] Certificado Origin CA e a lista de subdomínios prontos para colar no chat.

Cronograma

Relógio	Slides	Bloco	Min
00:00	1–7	VPS, comparação, por que não Kubernetes	16
00:16	8–14	Criar a VM (mão na massa)	35
00:51	15–21	Ubuntu: Docker, permissões, swap, SSH	32
01:23	—	Intervalo	10
01:33	22–26	Docker: imagem, Dockerfile, registry	16
01:49	27–35	Kamal: deploy.yml, accessory, segredos	26
02:15	36–44	CI/CD e OIDC — e configurar o OIDC	45
03:00	45–52	TLS, certificado, CA, Cloudflare	22
03:22	53–56	Primeiro deploy e operação	30
03:52	57–61	Prática, recapitulação, fim	10

Dá 4h02, e é otimista — assume que ninguém pega `NotAvailableForSubscription` e que o *subject* do OIDC acerta de primeira. Nenhuma das duas costuma acontecer.

Plano B, decidido de véspera

Escolha **agora** qual você vai usar:

B1 — Cortar o OIDC. Em vez do pipeline pelo Actions, cada aluno roda o Kamal da própria máquina:

```
GHCR_USER=... AZURE_VM_IP=... APP_HOST=... bundle exec kamal setup
```

Você explica o OIDC pelos slides (40–44, ~12 min) e mostra o workflow rodando **no seu** repositório.

Devolve 45 min e a aula fecha em 3h15. Eles fazem o pipeline em casa, com a apostila.

B2 — Virar demo a partir do intervalo. Se às 01:23 não houver pelo menos metade da turma com VM respondendo ao SSH, pare de esperar. Projete o seu deploy do começo ao fim e distribua o roteiro. **Melhor todo mundo ver funcionando do que metade travar no NSG.**

Bloco a bloco

00:00 — VPS (1–7)

Comece pela comparação (5) e vá para a decisão (6). A frase que resume a aula:

Escolher a ferramenta grande antes do problema grande é a forma mais cara de errar.

E o slide 7 é o que dá credibilidade: admita o custo. Uma VM é ponto único de falha, volume não é backup, o Postgres não é gerenciado.

00:16 — A VM (8-14)

Faça junto, passo a passo, projetado.

- **9** — cotas. Mostre a tela de Uso + quotas antes de tentar criar. Se alguém pegar `NotAvailableForSubscription`, é aqui que se resolve, não depois.
- **11** — **pare neste slide.** "Portas públicas: Nenhuma" é a escolha mais importante da tela. Diga por quê: o assistente, se deixado, libera SSH para a internet inteira, e bots varrem a porta 22 do IPv4 todo, o dia todo.
- **12-13** — chave pública e privada. É o conceito que também explica o JWT da Aula 3 e o TLS que vem daqui a pouco. Não pule.
- **14** — o `chmod 600`. Diga que o SSH **recusa** usar chave com permissão frouxa, e que é a primeira coisa a conferir quando der `Permission denied (publickey)`.

Ponto de decisão às 00:51: se menos da metade conectou por SSH, aplique o plano B2.

00:51 — Ubuntu (15-21)

- **17** — root e permissões. Rápido, mas não pule: a Aula 4 inteira depende disso.
- **18** — variável de ambiente. É o conceito que sustenta o `env.clear` / `env.secret` do Kamal.
- **20** — swap. Explique o OOM killer: numa VM de 1 GiB, Rails + Postgres + build estouram a RAM e o kernel mata o processo que estiver na frente, com uma mensagem que não explica nada.
- **21** — endurecer o SSH. **Fale isto em voz alta:** *"mantenham a sessão atual aberta e testem em outro terminal. Errar a config do SSH com uma sessão só é o jeito clássico de perder a máquina."*

01:33 — Docker (22-26)

Conceitual e rápido. Imagem é receita, container é bolo.

No **25**, as duas decisões: multi-stage (compilador não vai para produção) e usuário não-root (*"é uma linha que muda o tamanho do estrago"*).

01:49 — Kamal (27-35)

Abra o `config/deploy.yml` projetado e percorra junto com os slides.

O slide 31 (`clear` × `secret`) tem o argumento concreto: `JWT_SECRET` no `clear` apareceria em `docker inspect` e no histórico do shell. Se der, mostre um `docker inspect` de verdade.

O **34** (três camadas de segredo) é o slide que amarra tudo. Deixe na tela enquanto explica.

02:15 — CI/CD e OIDC (36–44)

O bloco mais perigoso do dia.

Explique a sequência (39) **antes** de mexer no portal, para eles saberem o que estão construindo: abre a 22 só para o IP do runner, faz o deploy, e fecha — inclusive se falhar.

Depois configurem juntos. **Avise antes**: o *subject* da credencial federada erra na primeira tentativa quase sempre.

```
repo:SEU-USUARIO/SEU-REPO:ref:refs/heads/main
```

Se aparecer `AADSTS700213`, é isso: confira maiúsculas, nome do repositório e branch, caractere por caractere.

E a atribuição de função vai **no NSG**, não na assinatura (44). Insista: se a credencial vazar, o estrago se limita a regras de firewall daquela máquina.

03:00 — TLS (45–52)

- **46–48** — HTTPS é HTTP num túnel; handshake em três passos; certificado e cadeia de confiança.
- **51** — os três modos. A frase que fixa: "com `Flexible`, o cadeado aparece e a senha do usuário trafega em claro no último trecho. É pior que não ter HTTPS, porque mente."
- **52** — por que não Let's Encrypt: com o DNS proxied, o desafio HTTP-01 não chega.

Aqui você distribui o certificado Origin CA wildcard que preparou.

03:22 — O primeiro deploy (53–56)

```
GHCR_USER=... AZURE_VM_IP=... APP_HOST=... bundle exec kamal config
```

Valida sem tocar em nada. **Faça isso antes** — pega variável faltando de graça.

Depois: **Actions** → **Deploy (Kamal)** → **Run workflow** → **command**: `setup`.

O `setup` demora. Use os ~8 minutos para o bloco de backup (56) e para responder pergunta.

E então, o momento da aula:

```
curl https://seunome.capacita.<domínio>/api/v1/status
```

03:52 — Fecho (57–61)

O slide 60 (`O QUE O SEEM-BACKEND TEM A MAIS`) é o convite: "nada disso é difícil depois do que vocês viram hoje. O código está lá, e agora vocês conseguem ler."

Perguntas que vão aparecer

Pergunta	Resposta curta
"Por que não Heroku/Render/Vercel? É mais fácil."	É mesmo, e para muita coisa é a escolha certa. Aqui o objetivo é vocês entenderem o que essas plataformas fazem por vocês — e o custo delas cresce rápido.
"E se a VM cair?"	Cai o sistema. É o custo declarado desta arquitetura. Com backup, você sobe outra em 20 min.
"Por que o banco não é gerenciado?"	Custo. Um Postgres gerenciado na Azure custa mais que a VM inteira. Para 500 usuários numa semana, não se paga.
"Preciso pagar pelo domínio?"	Nesta capacitação não — vocês usam um subdomínio meu. Fora daqui, ~R\$15/ano, ou grátis pelo GitHub Student Pack.
"O crédito da Azure vai acabar?"	São US\$100. Uma B1s ligada 24h custa ~US\$8/mês. Dá quase um ano. Desligue a VM quando não estiver usando.
"Por que abrir e fechar a porta 22 a cada deploy?"	Porque o IP do runner muda a cada execução, e deixar a 22 aberta para a internet é o que os bots procuram.
"Docker Compose não resolveria?"	Resolve subir os containers, sim. Não resolve build remoto, zero-downtime, rollback nem gestão de segredo. É o que o Kamal adiciona.
"Posso usar isso num projeto meu?"	Pode, e é a intenção. Troque o nome do serviço, o domínio e os secrets. O resto é igual.

Depois da capacitação

Mande no grupo:

1. **Desliguem a VM** quando não estiverem usando (`Parar` no portal — parada, ela não consome crédito de computação).
2. O link do `seem-backend`, com o convite do slide 60.
3. `docs/troubleshooting.md` — é o que eles vão abrir quando algo quebrar sem você por perto.